

The artifact that I'm using is my project for CS-340, a Client/Server Development class I took at SNHU. I included a CRUD method developed in python to help read from my database in MongoDB, and display this database in a user friendly dashboard using Jupyter notebooks and Python Dash, which allows users to view the data table, sort it, and show a pie chart and geo location of their chosen pet. The database I am using has about 10,000 entries in it, perfect for an enhancement based on databases. I am using my CS-340 dashboard and database project because it is a complete, working full stack app. The project includes a MongoDB database, a Python data access class, and an interactive Dash interface. The original version of the project worked, but there were several areas where I felt I could make meaningful improvements, especially with database performance, design, and security. Instead of only adding new features, I wanted to improve the way the existing application worked and show how much I have learned about database development.

One of the main areas I improved was how the application handled database credentials. In the original AnimalShelter class, the program checked for credentials but still used a hard coded connection string that contained the database password. I changed this so that the username, password, host, and optional port are read from environment variables instead. I also made sure the credentials are properly hidden before being used in the connection URL. If a required credential is missing, the program can identify what is missing without displaying the actual values. This was an important improvement because it made the application safer and removed sensitive information from the source code.

I also improved the way the database handles queries. I looked at the filters used by the dashboard and noticed that they commonly search for an exact sex value, specific breeds, and an age range. Based on these query patterns, I created indexes that are designed around the way the application actually searches the database. Rather than creating an index on every field, I focused on the fields that are used

by the rescue filters. This helped me understand that database indexes should be created based on how data is actually being queried.

Another major improvement was moving more of the filtering work into MongoDB. The original version pulled the entire dataset into Pandas and then performed the filtering in Python. I changed this by creating a `read_rescue_filter` method that uses a MongoDB `$match` and `$project` aggregation pipeline. This allows MongoDB to filter the records and calculate values such as age in weeks and years before the data is sent to Python. This reduces the amount of data that has to be transferred and also reduces the amount of work my Python app has to do.

I also created a performance and verification script to compare the original and the enhanced version. The results showed a big difference. The original approach scanned all 10,000 records, while the indexed queries only examined 5 records for the Mountain filter and 6 records for the Disaster filter. These numbers matched the number of rows returned by the queries. The original approach also transferred about 3.8 MB of data from the database each time a filter was used, while the aggregation approach returned only a few kilobytes of matching records. The new approach also took about 0.04 to 0.05 seconds compared with about 0.37 to 0.65 seconds for the Pandas approach. The row counts and results matched between the two approaches, which helped confirm that the performance improvements did not change the actual results.

While working on the project, I also found and fixed a problem with the map class. The selected row index was being used with the full dataset instead of the filtered table. This meant that after filtering or sorting the data, the map could display the wrong animal. Fixing this issue helped me see how changes to one part of an app can affect other parts of the overall app.

These changes helped me meet three of the course outcomes: Outcome 3, Outcome 4, and Outcome 5.

For Outcome 3, I compared the original approach of filtering data in Pandas with the new approach of filtering data directly in MongoDB. I chose the database side approach because it avoids pulling the entire dataset into Python before filtering it. I also had to consider the trade offs involved with indexes. Indexes can make queries faster, but they also use storage and can make database writes more expensive. Because of this, I focused on the fields that were actually being used by the apps queries instead of creating unnecessary indexes.

For Outcome 4, I used several techniques and tools that are commonly used when working with databases. This included MongoDBs aggregation framework, the explain command with execution statistics, and a repeatable testing script. Using these tools is what allowed me to measure the changes I wanted to have in my enhancement instead of simply assuming that the new version was faster. This was useful because I could compare actual results from the original and improved versions of the application.

For Outcome 5, I focused on security by looking at the hard coded database password as a security problem. A password stored directly in source code could potentially be accessed by anyone who has access to the repository or its history. I removed the password from the code and moved the credentials into environment variables. I also treated the exposed password as compromised and rotated the password in MongoDB Atlas. In addition, I made sure special characters in the credentials are properly escaped when creating the connection URL. My overall outcome coverage plan has stayed the same, although the project also showed me that there are additional security improvements I could make in the future, such as adding input validation and user authentication to the dashboard.

The biggest thing I learned from this project is that performance improvements should be measured instead of assumed. Using explain allowed me to see the difference between a scan of 10,000

documents and an indexed query that only examined a few documents. Comparing the aggregation approach with the original Pandas approach also showed me that the results were the same while using less data and taking less time. I also learned more about why the order of fields in a compound index matters and how much work a database can handle for an application when filtering and calculations are done where the data is stored.

The project also taught me more about security in a practical way. Moving a password into an environment variable sounds like a simple change, but I ran into several things that I had to consider. Special characters in a password can cause problems with a connection string, and extra spaces or quotation marks in environment variables can also cause connection problems. I also learned that error messages need to be handled carefully so that they do not accidentally reveal sensitive information.

I faced a lot challenges while making these changes. One of the first problems I ran into was a missing import for `quote_plus`, which caused a `NameError`. This was a small mistake, but it showed me how one missing import can stop an otherwise correct change from working. I also had a problem with the MongoDB connection string because of a quoting mistake in the `MongoClient` call. The resulting error message was confusing at first, but reading through the message carefully helped me understand that the driver was interpreting part of the connection string as multiple hostnames, which has since been fixed.

Another challenge was an authentication error from MongoDB Atlas. The cluster itself was reachable, but Atlas returned a bad auth error. This showed me that getting an application to work isn't just about the code within my project. The environment configuration, credentials, and connection settings also have to be correct. It was a good reminder that an application can work in one environment and fail in another if the configuration is different.

I also had an issue where the Water rescue filter returned zero results. At first, I thought the filter values might have been wrong. After checking the database, however, I found that there simply were no

records that matched those criteria. Both the original and new approaches correctly returned an empty result. This showed me to not immediately assume that an empty result means the code is broken. Sometimes the data itself is the reason for the result, and the application needs to handle that situation correctly.